56262

Kolmogorov-Arnold Networks 4

KAN-AD: Time Series Anomaly Detection with Kolmogorov–Arnold Network, ICML (2025)

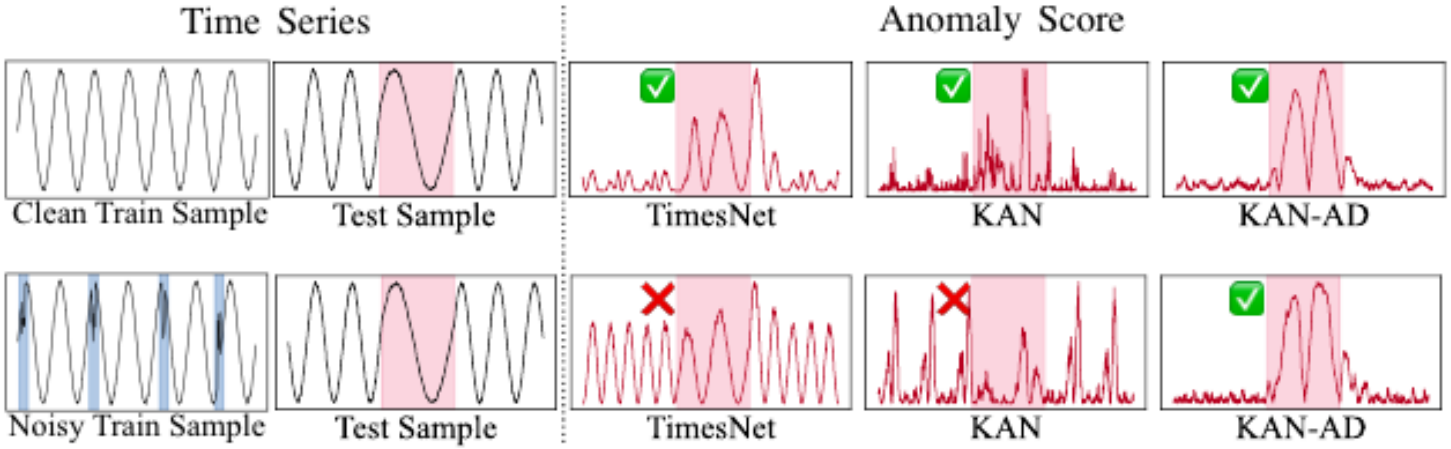
LESA: Learnable Stage-Aware Predictors for Diffusion Model Acceleration, CVPR (2026)

56262

KAN-AD: Time Series Anomaly Detection with Kolmogorov–Arnold Network, ICML (2025)

Motivation

- **Drawbacks of existing approaches**
  - Existing forecasting-based TSAD methods try to fit every detail in a time series, including tiny local peaks and drops.
  - KAN can specify univariate functions, i.e., B-spine function, these functions are not specifically designed for time series and can still overfit local features.



Comparison of anomaly detection performance

Basic Idea

- **KAN-AD**
  - Time Series Anomaly Detection (TSAD) models that are robust to noisy local fluctuations while remaining lightweight
  - KAN-AD, built meticulously on the KAN backbone, exhibits substantial improvements in both detection precision and inference efficiency.
  - KAN-AD replaces the B-spine function with Fourier series.

Problem Statement

- **Uivariate Time Series (UTS) anomaly detection**
  - The observed time series is defined as:

$$x_{0:t} = \{x_0, x_1, x_2, \ldots, x_t\}$$

  - The anomaly labels are defined as:

$$C = \{c_0, c_1, c_2, \ldots, c_t\}, \quad c_i \in \{0, 1\}$$

  - The objective of anomaly detection is : Given all historical observations before time i, predict whether the current point is anomalous.

$$[x_0, x_1, \ldots, x_i] \rightarrow c_i$$

KAN

- **Kolmogorov–Arnold Networks**
  - Any multivariate continuous function can be represented as a finite sum of compositions of univariate functions.

$$f(x_1, x_2, \ldots, x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^{n} \phi_{q,p}(x_p) \right)$$

- **Problem with Vanilla KAN**
  - Original KAN uses **B-spline functions** for the univariate functions.

B-splines are:

- highly localized,
- very flexible,
- capable of fitting small local structures.

KAN-AD 1/2

- **Core Idea**
  - Instead of directly predicting the raw time series, KAN-AD decomposes the signal into smooth basis functions.

$$f(x_{0:i}) = A_0 + \sum_{n=1}^{N} (A_n \cos(n x_{0:i}) + B_n \sin(n x_{0:i})) + \epsilon$$

- A_0: mean component
- A_n, B_n: learnable coefficients
- sine/cosine terms: smooth periodic basis functions
- ϵ: local noise/disturbance

KAN-AD 2/2

- **Design objective**
  - KAN-AD reconstruct a smooth approximation of the normal signal

$$x'_{0:i} = A_0 + \Theta(x_{0:i}) \times H$$

    - H: matrix of sine/cosine basis functions
    - $\Theta(x_{0:i})$: learned coefficients

- **Three stages**
  - Mapping Phase
  - Reducing Phase
  - Projection Phase

Three stages 1/3

- **Mapping Phase**
  - The mapping phase transforms the original time series signal into multiple new sets of values

$$x_{0:i} \in \mathbb{R}^T$$

$$\downarrow$$

$$x_{0:i} \in \mathbb{R}^{T \times (N+N)}$$

$$\mathrm{X} = x_{0:i}$$

$$\mathrm{S_n} = \{\sin(nx_{0:i}), \cos(nx_{0:i})\}$$

$$\mathrm{P_n} = \{\sin(\frac{2\pi ni}{T}), \cos(\frac{2\pi ni}{T})\}$$

- sine components
- cosine components
- periodic wave components

➡

```
raw signal
→ low-frequency trends
→ periodic patterns
→ oscillatory structures
```

Three stages 2/3

- **Reducing Phase**
  - After mapping, the model now has many function channels. The reducing phase learns how to combine them.
  - Univariate Function Matrix

$$\mathbf{H}^{(0)} = \text{Stack}(X, S_1, P_1, \ldots, S_n, P_n), \forall n \in [1, \ldots, N]$$

  - CNN-based Coefficient Learning

$$\mathbf{H}^{(l)} = \text{CNN}(\text{CNN}(\mathbf{H}^{(l-1)})) \quad \forall l \in [1, 2, \ldots, L]$$

  - BatchNorm + GELU

$$\text{CNN}(\mathbf{H}) = \text{GELU}(\text{BN}(\text{Conv}(\mathbf{H})))$$

```
raw signal
→ low-frequency trends
→ periodic patterns
→ oscillatory structures
```

➡

```
many Fourier features
→ coefficient estimation
→ smooth normal signal reconstruction
```

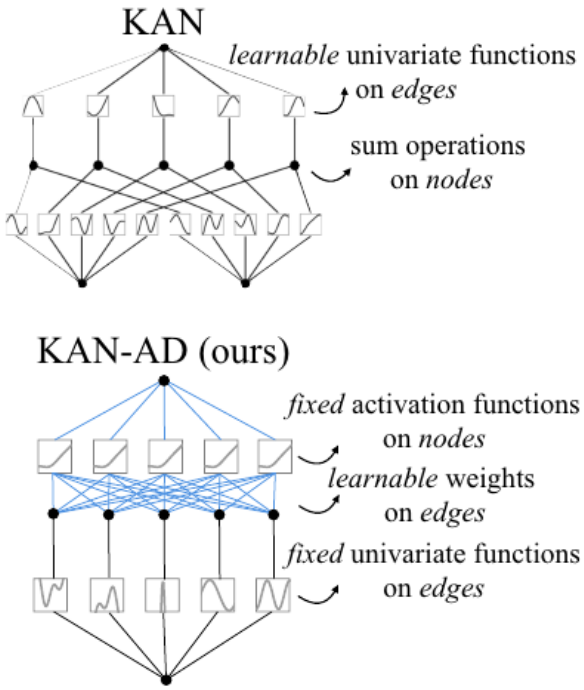
Three stages 3/3

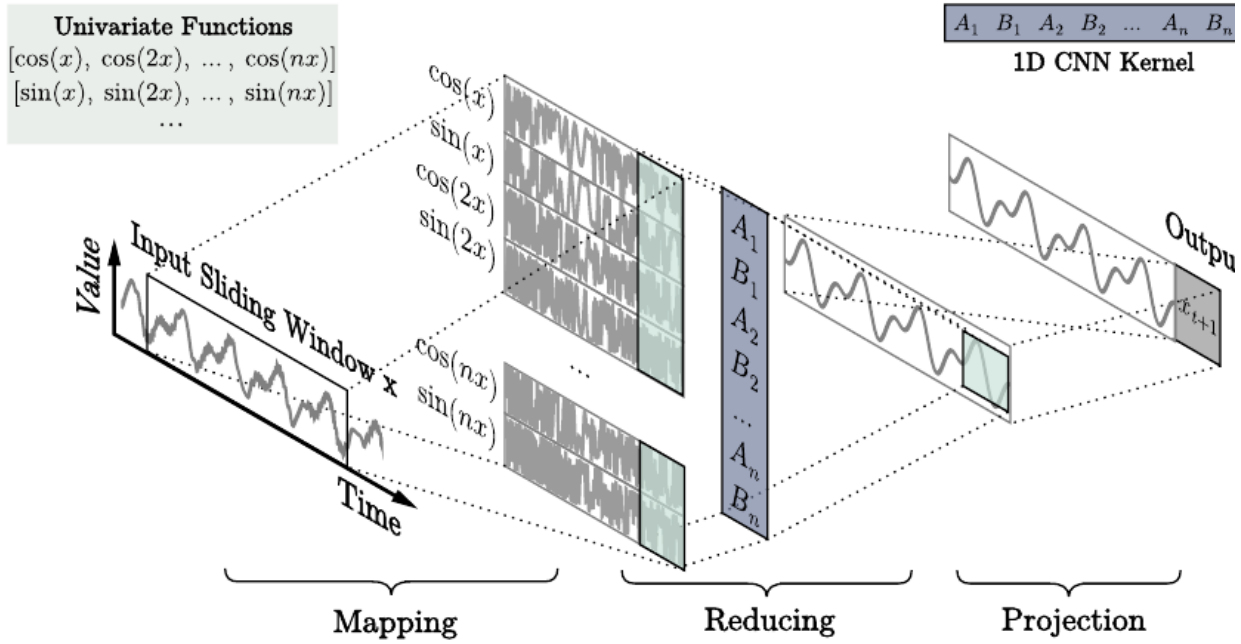
- **Reducing Phase**
  - The reconstructed normal pattern predicts the next value through a single-layer MLP.

$$x_{i+1} = W \cdot x'_{0:i} + b$$

- **Anomaly Detection**
  - During inference, KAN-AD compares predicted normal value with actual observed value.
  - Large difference ⇒ anomaly

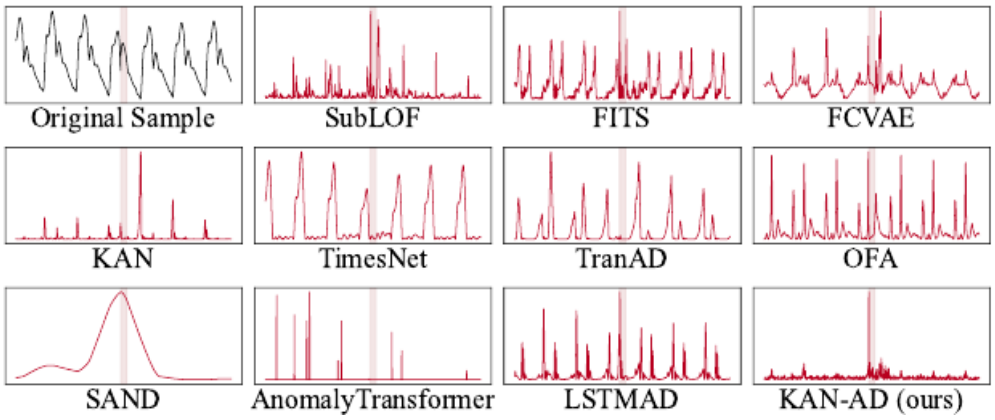
Illustration of KAN-AD





Performance Comparison

Method	KPI				TODS				WSD				UCR				Avg $F1_e$
	F1	$F1_e$	$F1_d$	AUPRC	F1	$F1_e$	$F1_d$	AUPRC	F1	$F1_e$	$F1_d$	AUPRC	F1	$F1_e$	$F1_d$	AUPRC	
SRCNN	0.4137	0.0994	0.2266	0.3355	0.6239	0.1918	0.4399	0.6076	0.4092	0.1185	0.1951	0.3080	0.5964	0.1369	0.1656	0.5109	0.1367
SAND	0.2710	0.0397	0.1097	0.2022	0.5372	0.1879	0.5103	0.5145	0.1761	0.0839	0.1267	0.1238	0.7044	0.5108	0.5116	0.6550	0.2056
AnoTrans	0.6103	0.3020	0.3623	0.5676	0.4875	0.1915	0.2918	0.4148	0.4348	0.2311	0.1517	0.3527	0.6135	0.1696	0.1084	0.5458	0.2236
TranAD	0.7553	0.5611	0.6399	0.7399	0.5035	0.2460	0.3619	0.4501	0.7570	0.6338	0.4158	0.7106	0.5278	0.1840	0.1554	0.4599	0.4062
SubLOF	0.7273	0.2805	0.4994	0.7015	0.7997	0.4795	0.7169	0.7809	0.8683	0.6585	0.4917	0.8353	0.8468	0.4772	0.4151	0.8001	0.4739
TimesNet	0.8022	0.6363	0.6995	0.8166	0.6232	0.3327	0.4495	0.6031	0.9406	0.8444	0.6170	0.9376	0.5273	0.1805	0.1439	0.4536	0.4985
FITS	0.9083	0.6353	0.8175	0.9359	0.7773	0.5416	0.6312	0.7725	0.9732	0.8391	0.6535	0.9771	0.6664	0.2926	0.2912	0.5969	0.5772
OFA	0.8810	0.6150	0.7952	0.9009	0.6928	0.5811	0.5588	0.7206	0.9564	0.8344	0.6250	0.9615	0.6294	0.3176	0.1503	0.5699	0.5870
FCVAE	0.9398	0.7556	0.8624	0.9572	0.8652	0.6995	0.7482	0.8798	0.9650	0.8610	0.6583	0.9653	0.7651	0.3812	0.2857	0.7145	0.6743
LSTMAD	0.9376	0.7742	**0.8782**	0.9624	0.8633	0.6981	0.7655	0.8740	0.9866	**0.9028**	**0.6743**	0.9849	0.7040	0.3482	0.3121	0.6432	0.6808
KAN	0.9411	0.7816	0.8666	0.9664	0.8109	0.6466	0.7518	0.8286	0.9879	0.8939	0.6650	**0.9881**	0.8016	0.4120	0.3971	0.7489	0.6835
KAN-AD	**0.9442** ±0.0007	**0.7989** ±0.0054	0.8755 ±0.0024	**0.9693** ±0.0008	**0.9425** ±0.0040	**0.8940** ±0.0022	**0.8391** ±0.0055	**0.9716** ±0.0035	**0.9888** ±0.0005	0.8927 ±0.0025	0.6623 ±0.0022	0.9868 ±0.0009	**0.8554** ±0.0040	**0.5335** ±0.0046	**0.5177** ±0.0042	**0.8188** ±0.0041	**0.7798**



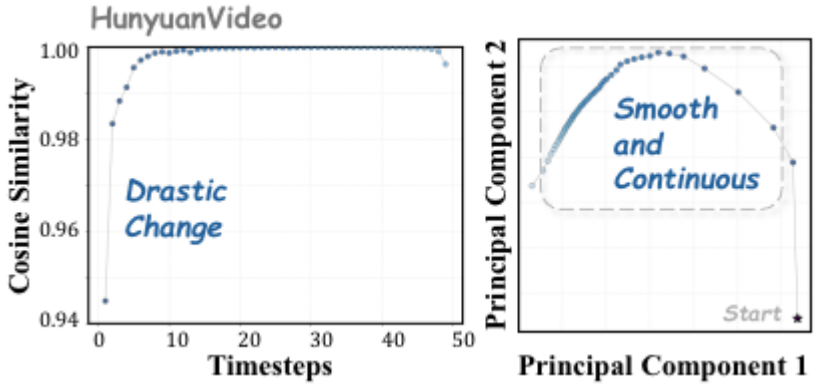
Method	GPU Time	CPU Time	Parameters	$F1_e$
SAND	-	5637s	-	0.5108
SubLOF	-	299s	-	0.4772
OFA	220s	3087s	81.920 M	0.3176
AnoTrans	201s	1152s	4.752 M	0.1696
FCVAE	2327s	1743s	1.414 M	0.3812
TimesNet	182s	259s	73,449	0.1805
LSTMAD	73s	267s	10,421	0.3482
KAN	66s	34s	1,360	0.4120
FITS	**32s**	**17s**	624	0.2926
TranAD	113s	62s	369	0.1840
KAN-AD	42s	36s	**274**	**0.5335**

56262

LESA: Learnable Stage-Aware Predictors for Diffusion Model Acceleration, CVPR (2026)

Motivation

- **Drawbacks of existing approaches**
  - Diffusion models are computationally expensive because they require many denoising steps involving a large computation.
  - Previous diffusion acceleration methods either reuse previous features or predict them using simple mathematical extrapolation.
  - Diffusion features evolve continuously and diffusion feature evolution is NOT globally smooth.



- early timesteps change drastically,
- middle timesteps evolve smoothly,
- late timesteps focus on detail refinement.



Basic Idea

- **LESA**
  - Diffusion dynamics are strongly stage-dependent.
  - LESA learns the temporal dynamics of the diffusion process for different diffusion stages
- **Stage-Aware Design**
  - The diffusion trajectory is divided into three stages:

Stage	Characteristic
High-noise	rapid feature change
Mid-noise	smooth evolution
Low-noise	detail refinement

Each stage uses:

- a dedicated predictor,
- and a different temporal window size.

Methodology 1/2

- **Predictor**
  - Input: previous K timestep features
  - Output: predicted future feature
- **Step 1: Linear Projection**
  - This fuses previous features into a latent representation.

$$z = W[h_{t+K-1}, h_{t+K-2}, ..., h_t] + b$$

- **Step 2: Timestep Modulation with KAN**
  - The timestep information is processed using a KAN:

$$\alpha = f_{KAN}(\Delta t_{L-1}, ..., \Delta t_0)$$

Methodology 2/2

- **Step 2: Timestep Modulation with KAN**
  - Why Use KAN?
    - Diffusion dynamics are nonlinear, irregular, stage-dependent.
    - Fixed nonlinearities are insufficient
    - KAN instead learns the activation functions themselves:

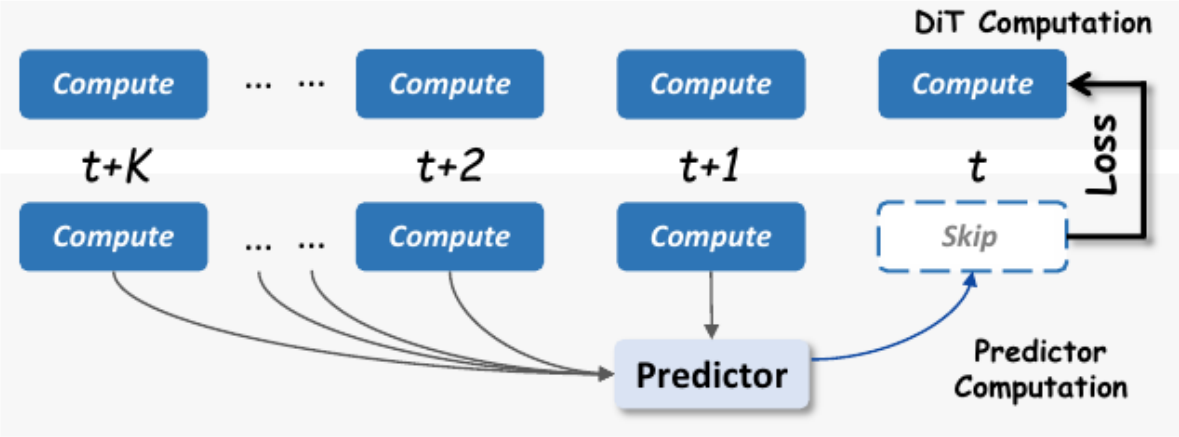
$$\alpha = f_{\mathrm{KAN}}(\Delta \mathbf{t}) = \sum_{m=1}^{M} w_m \, \phi_m\left(a_m^{\top} \Delta \mathbf{t}\right),$$

- **Step 3: Feature Prediction**
  - The predicted feature is generated using a residual update:

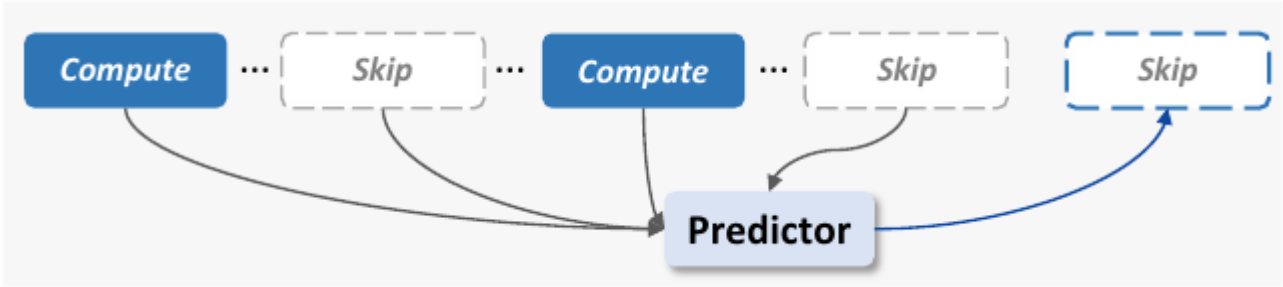
$$\hat{h}_{t-1} = h_t + \alpha z$$

- start from the current feature,
- then apply a learned temporal correction.

Training and Inference



Training Progress of the Predictor



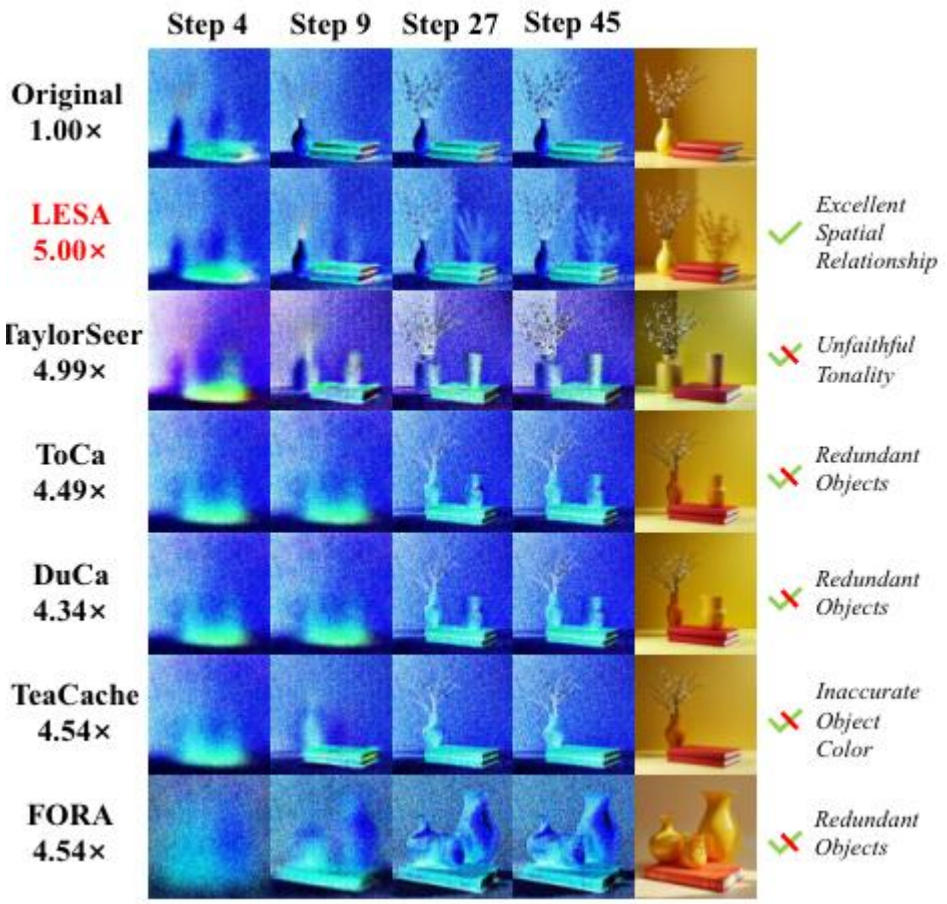
Inference Process with the Predictor

A shorter window ($K = 4$) is employed in the high-noise phase to accommodate its rapid and weakly correlated transitions, whereas a longer window ($K = 8$) is utilized in the subsequent, more stable phases to better capture the intricate, history-dependent evolution of features.

Text-to-Image Generation 1/2

Method	Acceleration				Quality Metrics		Perceptual Metrics		
FLUX	Latency(s) ↓	Speed ↑	FLOPs(T) ↓	Speed ↑	Image Reward↑	CLIP Score↑	PSNR↑	SSIM↑	LPIPS↓
[dev]: 50 steps	23.24	1.00×	3726.87	1.00×	0.99	32.64	∞	1.00	0.00
50% **steps**	11.82	1.97×	1863.44	2.00×	0.97	32.57	29.56	0.73	0.31
PAB	17.84	1.30×	3013.13	1.24×	0.95	32.55	28.84	0.67	0.40
DBCache	16.88	1.38×	2384.29	1.56×	1.01	32.53	33.86	0.87	0.12
FORA ($\mathcal{N}$=3)	9.06	2.57×	1267.89	2.94×	0.99	32.27	30.65	0.77	0.25
TeaCache (l=0.6)	9.13	2.55×	1342.20	2.78×	0.91	32.11	29.03	0.68	0.40
TaylorSeer ($\mathcal{N}$=4, O=2)	8.78	2.64×	1118.86	3.57×	**1.02**	32.41	29.87	0.73	0.30
LESA ($\mathcal{N}$=5)	**6.57**	**3.54**×	**969.11**	**3.85**×	1.00	**32.70**	**30.96**	**0.77**	**0.24**
FORA($\mathcal{N}$=5) †	5.97	3.89×	820.80	4.54×	0.82	32.48	28.44	0.60	0.50
ToCa($\mathcal{N}$=8, $\mathcal{R}$=75%)	12.39	1.88×	829.86	4.49×	0.95	32.60	29.07	0.64	0.43
DuCa($\mathcal{N}$=8, $\mathcal{R}$=70%)	9.40	2.47×	858.27	4.34×	0.94	32.58	29.06	0.64	0.43
TeaCache(l=1.0) †	7.07	3.29×	820.55	4.54×	0.84	31.88	28.61	0.64	0.48
TaylorSeer($\mathcal{N}$=6, O=2)	6.73	3.45×	746.28	4.99×	**1.02**	32.53	28.94	0.66	0.40
LESA ($\mathcal{N}$=7)	**5.19**	**4.48**×	**745.51**	**5.00**×	0.98	**32.88**	**30.17**	**0.72**	**0.32**
FORA($\mathcal{N}$=7) †	5.09	4.57×	597.25	6.24×	0.68	31.90	28.32	0.59	0.54
ToCa($\mathcal{N}$=12, $\mathcal{R}$=85%) †	9.82	2.37×	618.57	6.02×	0.80	32.32	28.70	0.60	0.52
DuCa($\mathcal{N}$=12, $\mathcal{R}$=80%) †	7.74	3.00×	646.97	5.76×	0.77	32.20	28.71	0.60	0.53
TeaCache(l=1.4) †	6.14	3.79×	671.51	5.55×	0.74	31.78	28.12	0.48	0.68
TaylorSeer($\mathcal{N}$=9, O=2) †	5.85	3.97×	597.25	6.24×	0.86	32.04	28.38	0.59	0.51
LESA ($\mathcal{N}$=10)	**4.28**	**5.43**×	**596.44**	**6.25**×	**0.91**	**32.65**	**29.65**	**0.67**	**0.40**
[schnell]: 4 steps	2.13	1.00×	278.41	1.00×	0.93	34.09	∞	1.00	0.00
LESA ($\mathcal{N}$=3)	1.31	1.63×	139.21	2.00×	0.95	34.48	30.31	0.80	0.16
LESA ($\mathcal{N}$=4)	1.15	1.85×	69.61	4.00×	0.91	34.67	29.21	0.69	0.30

Text-to-Image Generation 2/2



Experiment Settings			Quality Metrics		Perceptual Metrics		
Interval	Segmented	Module	Image Reward↑	CLIP Score↑	PSNR↑	SSIM↑	LPIPS↓
$\mathcal{N}=5$	✔	KAN	1.00	32.70	30.96	0.77	0.24
$\mathcal{N}=5$	✘	KAN	1.00	32.67	30.77	0.77	0.25
$\mathcal{N}=5$	✔	MLP	0.99	32.69	30.76	0.77	0.25
$\mathcal{N}=5$	✘	MLP	0.95	32.50	30.29	0.73	0.31
$\mathcal{N}=7$	✔	KAN	0.98	32.88	30.17	0.72	0.32
$\mathcal{N}=7$	✘	KAN	0.95	32.75	30.10	0.71	0.33
$\mathcal{N}=7$	✔	MLP	0.96	32.72	30.11	0.71	0.33
$\mathcal{N}=7$	✘	MLP	0.95	32.65	30.08	0.71	0.34
$\mathcal{N}=10$	✔	KAN	0.91	32.65	29.65	0.67	0.40
$\mathcal{N}=10$	✘	KAN	0.88	32.58	29.60	0.66	0.41
$\mathcal{N}=10$	✔	MLP	0.89	32.57	29.59	0.66	0.42
$\mathcal{N}=10$	✘	MLP	0.88	32.51	29.55	0.66	0.43

Text-to-Video Generation 1/2

Method HunyuanVideo	Acceleration				Perceptual Metrics		
	Latency(s) ↓	Speed ↑	FLOPs(T) ↓	Speed ↑	PSNR↑	SSIM↑	LPIPS↓
Original: 50 steps	98.91	1.00×	29773.0	1.00×	∞	1.00	0.00
22% **steps**	22.98	4.30×	6550.1	4.55×	17.65	0.59	0.42
ToCa($\mathcal{N}=5, \mathcal{R}=90\%$)	26.12	3.79×	7006.2	4.25×	17.04	0.54	0.44
DuCa($\mathcal{N}=5, \mathcal{R}=90\%$)	23.08	4.29×	6483.2	4.48×	17.08	0.54	0.43
TeaCache($l=0.4$)	21.83	4.53×	6550.1	4.55×	18.25	0.61	0.38
FORA($N=5$)	22.61	4.37×	5960.4	5.00×	17.00	0.53	0.44
TaylorSeer($\mathcal{N}=5, O=1$)	23.66	4.18×	5960.4	5.00×	17.29	0.55	0.42
SpeCa($\mathcal{N}_{\text{max}}=8, \mathcal{N}_{\text{min}}=2$)	23.22	4.26×	5692.7	5.23×	17.73	0.59	0.39
ClusCa($\mathcal{N}=5, O=1, K=16$)	24.35	4.06×	5373.0	5.54×	17.28	0.55	0.42
LESA($\mathcal{N}=7$)	19.37	5.11×	5960.4	5.00×	**21.43**	**0.72**	**0.29**
LESA($\mathcal{N}=8$)	**17.47**	**5.66**×	**5354.9**	**5.56**×	21.05	0.70	0.32

Text-to-Video Generation 2/2

